SOP & CORS

SOP: SAME-ORIGIN POLICY



SAME-ORIGIN POLICY

La Same-origin policy (SOP) è un meccanismo di sicurezza implementato nei browser. Limita la comunicazione tra script provenienti da **origini diverse**.



COSA PROTEGGE SOP

La Same-origin policy è una difesa fondamentale che protegge l'utente da codice potenzialmente malevolo iniettato in una pagina web. In pratica, impedisce che un sito web possa spiare i dati di un altro sito.

Gli obiettivi principali sono:

- **Prevenzione del Furto di Dati (Data Leak):** Impedisce a uno script (eseguito da un sito malevolo `malicious.com`) di leggere dati sensibili (come il saldo bancario, messaggi privati, o cookie di sessione) ottenuti tramite richieste AJAX/fetch inviate a un sito legittimo (`bank.com`). Lo script non può accedere al corpo della risposta HTTP.
- **Isolamento delle Applicazioni Web:** Assicura che la logica e i dati di un'applicazione siano isolati nel loro contesto di origine, impedendo interferenze dirette da parte di script di terze parti.
- **Mitigazione di CSRF (Limitata):** Anche se la SOP non impedisce l'invio di richieste "semplici" (che è la base degli attacchi CSRF), impedisce all'attaccante di ricevere e analizzare la risposta, rendendo gli attacchi più difficili da orchestrare e meno efficaci nel determinare l'esito.

ESEMPIO I - STESSA ORIGINE

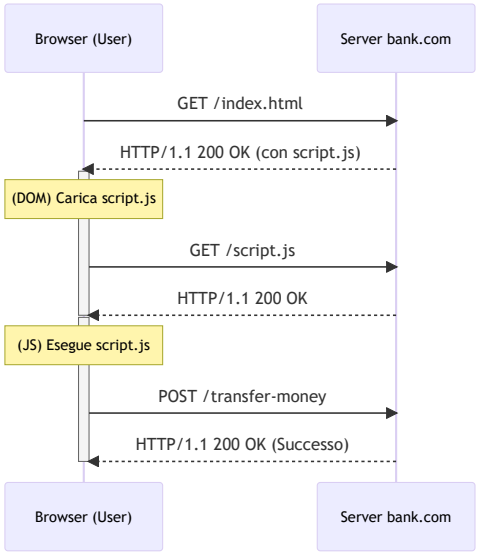
```
<!doctype html>
<html>
<head><!-- ... --></head>
<body>
<!-- ... -->
<script src="/script.js"></script>
</body>
</html>
```



SEQUENZA DI SCRIPT (ESEMPIO 1 - STESSA ORIGINE)

Questo diagramma mostra una sequenza in cui il browser (utente) riceve `index.html` da `bank.com`, carica `script.js` (dalla stessa origine), ed esegue una `POST` per trasferire denaro (sempre alla stessa origine `bank.com`). L'operazione ha successo perché tutto avviene all'interno della stessa origine.




Browser (User)
Server bank.com
GET /index.html
HTTP/1.1 200 OK (con script.js)
(DOM) Carica script.js
GET /script.js
HTTP/1.1 200 OK
(JS) Esegue script.js
POST /transfer-money
HTTP/1.1 200 OK (Successo)
Browser (User)
Server bank.com

ESEMPIO 2 - ORIGINE DIVERSA/ATTACCO

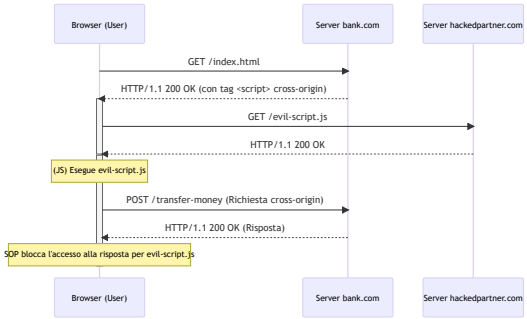
```
<!doctype html>
<html>
<head><!-- ... --></head>
<body>
<!-- ... -->
<script src="https://hackedpartner.com/evil-script.js">
</script>
</body>
</html>
```

SEQUENZA DI SCRIPT (ESEMPIO 2 - ORIGINE DIVERSA/ATTACCO)

- Il browser carica `index.html` da `bank.com`.
- Il browser carica `evil-script.js` da `hackedpartner.com`.
- Quando `evil-script.js` tenta di eseguire una richiesta `POST /transfer-money` a `bank.com`, la **Same-origin policy blocca l'accesso alla risposta**.

Il *browser* invia comunque la richiesta, ma lo *script malevolo* non può leggere la risposta.




Browser (User)
Server bank.com
Server hackedpartner.com
GET /index.html
HTTP/1.1 200 OK (con tag <script> cross-origin)
GET /evil-script.js
HTTP/1.1 200 OK
(JS) Esegue evil-script.js
POST /transfer-money (Richiesta cross-origin)
HTTP/1.1 200 OK (Risposta)
SOP blocca l'accesso alla risposta per evil-script.js
Browser (User)
Server bank.com
Server hackedpartner.com



L'ORIGINE

Due URL hanno la stessa origine se sono identici il **protocollo**, il **dominio** e la **porta**.

JRL Esempio	Protocollo	Host	Porta	Stessa Origine rispetto a http://site.com?
`http://site.com/page.html`	`http`	`site.com`	(default 80)	Sì
`https://site.com/page.html`	`https`	`site.com`	(default 443)	No (Protocollo diverso)
`http://www.site.com`	`http`	`www.site.com`	(default 80)	No (Host diverso)
`http://site.com:8080`	`http`	`site.com`	`8080`	No (Porta diversa)



COS'È BLOCCATO?

La SOP non impedisce l'invio della richiesta, ma impedisce allo script malevolo di **leggere la risposta** della richiesta HTTP.
Tuttavia, alcuni tag HTML possono comunque effettuare richieste cross-origin, come:

- `<script src="...">`
- `<link href="...">`
- `<img>`
- `<iframe>`



SOP: REGOLE IMPORTANTI

- **Le origini possono incorporare risorse cross-origin.** (Es. immagini, CSS, script).
- **Le origini possono innescare operazioni cross-origin.** (Es. un form che fa un submit a un'origine diversa).
- **Le origini non possono leggere dati cross-origin.**



COS'È PROTETTO DA SOP

SOP **protegge da lettura/manipolazione** da parte di codice malevolo:

1. **DOM (Document Object Model):**
    - **Cosa protegge:** Lo script su `evil.com` non può accedere al contenuto (testo, immagini, struttura) della pagina caricata da `bank.com`.
    - **Contesto:** Questo impedisce a un attaccante di leggere informazioni sensibili come numeri di conto o, nel caso del CSRF, di leggere il **Token CSRF** nascosto nel form.



1. **Cookies, HTTP Authentication, IndexedDB, Web Storage (LocalStorage/SessionStorage):**
    - **Cosa protegge:** Queste sono tutte forme di dati archiviati localmente nel browser, spesso utilizzati per mantenere lo stato di sessione o le preferenze dell'utente.
    - **Contesto:** Uno script su `evil.com` non può leggere il tuo cookie di sessione per `bank.com`, né accedere ai dati che `bank.com` ha salvato in LocalStorage. Questo è fondamentale per la sicurezza delle sessioni.
    - ***Eccezione Importante (Cookie):*** La SOP *protegge dalla **lettura*** del cookie tramite JavaScript (ad esempio `document.cookie`). Tuttavia, come abbiamo visto nella discussione sul CSRF, la SOP **NON** impedisce al browser di **allegare automaticamente** i cookie alle richieste cross-origin che invia.



Risorsa	Azione Proibita dalla SOP	Azione Permessa (Vulnerabile al CSRF)
DOM / Contenuto	Leggere l'HTML, il testo o i campi nascosti del sito target.	N/A
Cookie	Leggere il valore del cookie (`document.cookie`).	L'invio automatico del cookie con una richiesta forzata.
IndexedDB / Storage	Leggere o modificare i dati salvati dal sito target.	N/A



CORS: CROSS-ORIGIN RESOURCE SHARING

CORS (Cross-Origin Resource Sharing) è un meccanismo che permette ai server di specificare, tramite header HTTP, **quali origini possono accedere alle loro risorse**.

CORS è la soluzione per *allentare* le restrizioni imposte dalla Same-Origin Policy.



ESEMPIO DI CORS

Quando un browser vede una richiesta HTTP cross-origin, aggiunge l'header `Origin` alla richiesta.

Richiesta del browser:

Origin: **http://my-frontend.com**

Risposta del server (se permette la richiesta):

Access-Control-Allow-Origin: **http://my-frontend.com**

Se l'header `Access-Control-Allow-Origin` corrisponde all'origine della richiesta, il browser permette allo script di accedere alla risposta.



ACCESS-CONTROL-ALLOW-ORIGIN

- `Access-Control-Allow-Origin: http://my-frontend.com`
  - Permette solo a questa origine.
- `Access-Control-Allow-Origin: *`
  - Permette a **qualsiasi** origine (usare con cautela).



RICHIESTE SEMPLICI (SIMPLE REQUESTS)

Una richiesta HTTP è considerata *semplice* se:

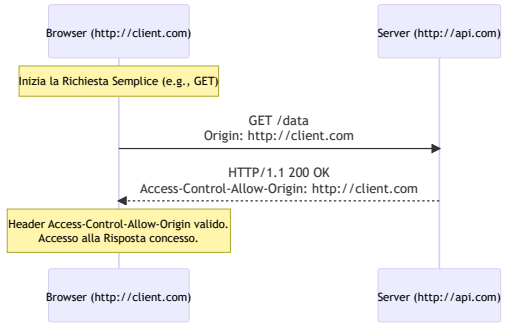
1. Usa uno dei metodi: `GET`, `HEAD`, `POST`.
2. Gli unici header personalizzati sono quelli che rientrano in una lista specifica (ad esempio: `Accept`, `Accept-Language`, `Content-Language`, `Content-Type`).
3. Il valore di `Content-Type` è uno tra: `application/x-www-form-urlencoded`, `multipart/form-data`, o `text/plain`.



RICHIESTE SEMPLICI (SIMPLE REQUESTS)

Se la richiesta è semplice, il browser invia immediatamente la richiesta e il server risponde con l'header `Access-Control-Allow-Origin`. Se l'header è presente e valido, lo script cross-origin può accedere alla risposta.




Browser (http://client.com)
Server (http://api.com)
Inizia la Richiesta Semplice (e.g., GET)
GET /data
Origin: http://client.com
HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://client.com
Header Access-Control-Allow-Origin valido.
Accesso alla Risposta concesso.
Browser (http://client.com)
Server (http://api.com)

RICHIESTE PREFLIGHT (PREFLIGHTED REQUESTS)

Una richiesta *preflight* è una richiesta che non soddisfa le condizioni per essere considerata semplice.
Richiede una richiesta HTTP `OPTIONS` preliminare, prima della richiesta effettiva (gestita automaticamente dal browser), per concordare la SOP con il server.



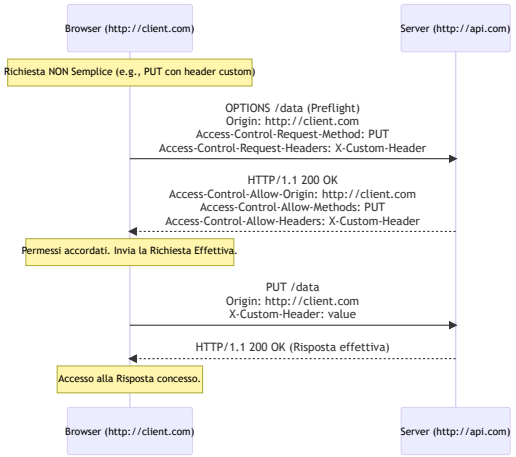
RICHIESTE PREFLIGHT (SCHEMA)

Il browser invia la richiesta OPTIONS con header come Access-Control-Request-Method e Access-Control-Request-Headers per chiedere il permesso.

Il server risponde con gli header Access-Control-Allow-* (metodi, headers, ecc.).

Se il permesso è accordato, il browser invia la richiesta effettiva.




Browser (http://client.com)
Server (http://api.com)
Richiesta NON Semplice (e.g., PUT con header custom)
OPTIONS /data (Preflight)
Origin: http://client.com
Access-Control-Request-Method: PUT
Access-Control-Request-Headers: X-Custom-Header
HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://client.com
Access-Control-Allow-Methods: PUT
Access-Control-Allow-Headers: X-Custom-Header
Permessi accordati. Invia la Richiesta Effettiva.
PUT /data
Origin: http://client.com
X-Custom-Header: value
HTTP/1.1 200 OK (Risposta effettiva)
Accesso alla Risposta concesso.
Browser (http://client.com)
Server (http://api.com)

RICHIESTE PREFLIGHT CON AUTENTICAZIONE

Se la richiesta include credenziali (come l'header `Authorization`), è necessario che:

- La richiesta `OPTIONS` includa l'header `Access-Control-Request-Headers: Authorization`.
- Il server risponda con l'header `Access-Control-Allow-Credentials: true`.

CSRF



COS'È IL CSRF?

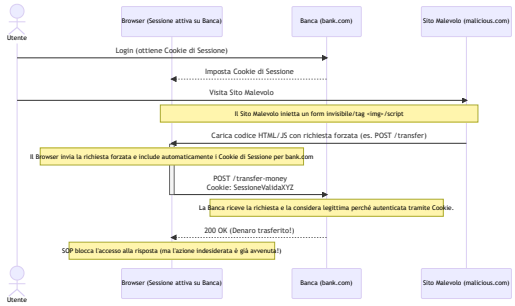
Il **Cross-Site Request Forgery (CSRF)**, a volte chiamato *sea-surf* o *XSRF*, è una vulnerabilità che induce l'utente finale a eseguire azioni indesiderate su un'applicazione web in cui è attualmente autenticato.

Il presupposto dell'attacco:

1. L'utente ha una sessione attiva (è loggato) su un sito legittimo (es. `bank.com`).
2. L'utente visita un sito malevolo (`malicious.com`) o apre una mail contenente codice malevolo.
3. Il codice malevolo innesca una richiesta HTTP (di solito un `GET` o un `POST` semplice) al sito legittimo (`bank.com`).
4. Il browser aggiunge automaticamente i cookie di sessione validi di `bank.com` alla richiesta.
5. Il server di `bank.com` esegue l'azione (es. cambio password o trasferimento di denaro) credendo provenga dall'utente.

L'ATTACCO CSRF (DIAGRAMMA)

L'attaccante sfrutta la fiducia del server nel browser dell'utente.



IL RUOLO DEI TRE ATTORI

L'attacco CSRF coinvolge tre attori principali:

1. L'Origine Malevola (L'Attaccante)

- **Cosa fa:** Ospita il *payload* dell'attacco. Questo payload è un semplice codice HTML (come un form invisibile, un tag `<img>`, o uno script) che istruisce il browser dell'utente a fare una richiesta al sito bersaglio (`bank.com`).
- **Esempio di Payload:** `<img src="https://bank.com/transfer?to=Attacker&amount=1000">`



2. Il Browser dell'Utente (La Vittima)

- **Cosa fa:**
  - Carica il codice HTML/JS dall'origine malevola.
  - Esegue l'istruzione del payload (es. tenta di caricare l'immagine o invia il form).
  - **Punto Critico:** Quando il browser esegue una richiesta HTTP verso un dominio (`bank.com`), verifica automaticamente se possiede cookie di sessione validi per quel dominio. Se l'utente è loggato, **il browser allega quei cookie alla richiesta**.
  - Il browser invia la richiesta forzata **completa di autenticazione**.



3. Il Server Bersaglio (Il Sito Legittimo)

- **Cosa fa:**
  - Riceve la richiesta, inclusi i cookie di sessione validi.
  - Controlla i cookie e determina che la richiesta è autenticata e proviene dall'utente legittimo (la vittima).
  - **Esegue l'azione**, credendo che l'utente abbia intenzionalmente richiesto l'operazione (es. il trasferimento di denaro).



CSRF VS. SOP (SAME-ORIGIN POLICY)

L'attacco CSRF sfrutta proprio il limite della SOP:

- **CSRF Sfrutta:** La SOP non impedisce affatto l'**invio** di richieste HTTP cross-origin (soprattutto se sono richieste semplici, come un GET o un POST standard di form). Sfrutta il fatto che il browser aggiunge i cookie per te.
- **SOP Protegge:** La SOP impedirebbe al codice sul sito malevolo di **leggere la risposta** del server bersaglio. Tuttavia, nell'attacco CSRF l'attaccante non ha bisogno di leggere la risposta; ha solo bisogno che l'azione (il trasferimento, la cancellazione, ecc.) venga eseguita. L'azione è già avvenuta prima che la SOP entri in gioco per bloccare la lettura del risultato.

In sintesi, il codice malevolo imposta il missile, ma è il browser dell'utente a premere involontariamente il pulsante di lancio, completo dei codici di autenticazione (i cookie).

LE RICHIESTE SFRUTTABILI

Il CSRF è più facile da eseguire quando l'azione indesiderata può essere eseguita tramite una **Richiesta Semplice** (Simple Request, vedere slide SOP/CORS), in particolare:

- **GET:** Molti vecchi sistemi usavano `GET` per azioni di stato (es. `GET /transfer?to=Attacker&amount=100`).
- **POST (via form submission):** Il browser può inviare un `POST` con `Content-Type: application/x-www-form-urlencoded` (il default dei form HTML) a qualsiasi origine, inclusi i cookie.

Nota: La SOP impedisce allo script malevolo di *leggere la risposta*, ma **non impedisce che la richiesta venga inviata e l'azione venga eseguita** sul server.



DIFESA #1: CSRF TOKEN SINCRONIZZATO (SYNCHRONIZER TOKEN PATTERN)

Il meccanismo di difesa più comune e robusto contro il CSRF è il **Synchronizer Token Pattern**.

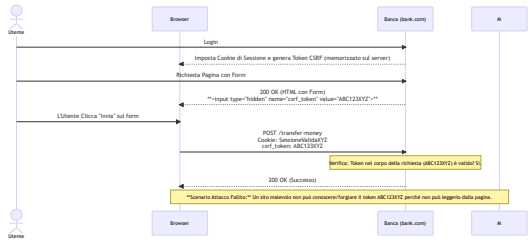
Idea: Per ogni richiesta che modifica lo stato (es. `POST`, `PUT`, `DELETE`), il server richiede un valore segreto (il *CSRF Token*) che deve essere inviato insieme ai cookie di sessione.

1. **Generazione:** Il server genera un token CSRF segreto, unico per la sessione dell'utente.
2. **Distribuzione:** Il server include questo token in un campo nascosto (`<input type="hidden">`) all'interno di tutti i form che modificano lo stato.
3. **Verifica:** Al momento della ricezione della richiesta, il server verifica che il token inviato nel corpo/header della richiesta **corrisponda** al token memorizzato lato server per quella sessione.



IL FLUSSO DI DIFESA CON CSRF TOKEN

Un attaccante, operando da un'origine diversa, non può né leggere il contenuto della pagina web legittima (per estrarre il token), né indovinare il valore segreto del token.



Metodo preferito?

DIFESA #2: CORS PREFLIGHT E HEADER `AUTHORIZATION`

Nel contesto dei servizi web basati su API REST/JSON (come spesso accade nel WASA Project), le richieste non semplici (es. `PUT`, `DELETE`, o `POST` con `Content-Type: application/json` o header custom come `Authorization`) **richiedono un Preflight CORS**.

Perché funziona come difesa CSRF?

1. La richiesta forzata CSRF (proveniente da un form HTML o un semplice tag) **non può includere header custom** come `Authorization`.
2. Se l'applicazione *richiede* l'header `Authorization` per l'autenticazione, una richiesta CSRF non autenticata fallirà.
3. Se si usa la protezione con header custom o `Content-Type` non standard (`application/json`), il browser invia un `OPTIONS` Preflight, che **non può essere forgiato** da un semplice form HTML in un attacco CSRF classico.



NON È NECESSARIO IL CSRF TOKEN

Nel progetto si utilizza l'header `Authorization` per l'autenticazione/autorizzazione.
Dato che l'header `Authorization` fa scattare una richiesta preflight, **non è necessario** implementare un token CSRF (Cross-Site Request Forgery) aggiuntivo, a condizione che CORS sia configurato correttamente.



LINK UTILI

- https://developer.mozilla.org/en-US/docs/Web/Security/Same_origin_policy
- https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS
- https://www.youtube.com/watch?v=KaEj_qZgiKY
- https://en.wikipedia.org/wiki/Same-origin_policy
- https://en.wikipedia.org/wiki/Cross-origin_resource_sharing

